

Bangkok CCTV Vehicle Detection

Take-Home Midterm Exam Report

2110531 Data Science and Data Engineering Tools (2026/1)

Pongsatorn Kanjanasantisak, student ID 6970180821

Kaggle team 6970180821_Pongsatorn

Result: mAP@50 **0.707** on the Kaggle public leaderboard (1st place at the time of writing).

Contents

1. Introduction

2. Data preparation

- 2.1 Dataset statistics
- 2.2 Validation split by camera
- 2.3 Cleaning
- 2.4 Class re-balancing by oversampling
- 2.5 Augmentation and input size
- 2.6 Crops for the classifier
- 2.7 External data

3. Model

- 3.1 Pipeline
- 3.2 Models
- 3.3 Training settings
- 3.4 The classifier adds a second guess
- 3.5 Generative AI and cost

4. Results

- 4.1 What each stage adds
- 4.2 Epochs and model size
- 4.3 Kaggle submissions
- 4.4 External data
- 4.5 Per-class results

5. Discussion

- 5.1 Error analysis
- 5.2 Lessons from the process
- 5.3 How to improve

6. Conclusion

Appendix: reproduction and submission package

1. Introduction

The task is to detect and classify vehicles in Bangkok traffic-camera images. The images come from the Bangkok Metropolitan Administration (BMA) traffic platform, one frame every 30 minutes from 25 to 31 August 2026. Every vehicle must be found with a bounding box and assigned one of 8 classes: Car, Motorcycle, Bus, Truck, Tuktuk, Van, Pickup and Songthaew.

Submissions are scored on Kaggle with **mAP@50**: average precision at IoU 0.5, computed per class with pycocotools and averaged over the 8 classes. Every class counts equally, so the rare classes (Songthaew, Tuktuk, Van, Pickup) matter as much as Car and Motorcycle.

Two properties of the data shape the whole solution:

- **The test cameras are new.** The 5 test cameras never appear in the training data, so a model must work on camera views it has not seen.
- **The objects are tiny and the classes are unbalanced.** Images are 352×288 px, most vehicles are under 32×32 px, and Car plus Motorcycle make up 89% of all boxes.

The final solution is an ensemble of two detectors (RF-DETR Large and YOLO26m) with flip test-time augmentation, Weighted Boxes Fusion, and a ConvNeXt-Tiny crop classifier that adds a second class guess for easily confused vehicles. It scores **0.707** mAP@50 on the public leaderboard. Every design choice was made on a validation set of held-out cameras, not on the public score.

2. Data preparation

2.1 Dataset statistics

Item	Value
Image size	352 × 288 px (all images)
Train images	2,991 (2,889 with boxes, 102 without any box)
Test images	1,013 (997 listed in sample_submission.csv and scored)
Labelled boxes	27,396 (average 9.5 per image, maximum 48)
Cameras	15 in train; 5 different ones in test (1068, 1072, 1192, 1439, 227)

ID	Class	All train boxes	Train split	Val split
0	Car	17,762	15,282	2,480
1	Motorcycle	6,563	5,172	1,391
2	Bus	482	415	67
3	Truck	1,356	1,072	284
4	Tuktuk	265	221	44
5	Van	354	323	31
6	Pickup	497	383	114
7	Songthaew	117	93	24
	Images / cameras	2,991 / 15	2,388 / 12	603 / 3

Table 2.1 Class counts and the camera-held-out split.

Observations that drove the design:

- **Objects are very small.** 80% of cars and 97% of motorcycles are under 32×32 px; the median motorcycle box is 7×17 px.
- **Classes are very unbalanced.** Songthaew has 117 boxes, Tuktuk 265; Car has 17,762.
- **Songthaew comes mostly from one camera:** 91 of its 117 boxes are from camera 1426.

2.2 Validation split by camera

Because the test cameras are new, validation also uses unseen cameras: three whole cameras (**1066, 1427, 172**) are held out. A random image split would put near-identical frames of the same camera on both sides and overstate the score. The three cameras keep some of every rare class in validation (Songthaew 24, Tuktuk 44, Van 31) while camera 1426, with most Songthaews, stays in training.

2.3 Cleaning

- Box coordinates are clipped to the 352×288 image, and boxes that become empty are removed.
- The 102 images without boxes are kept as background examples, so the models learn what an empty road looks like.
- Train and test images were checked against the current Kaggle download (identical files).

2.4 Class re-balancing by oversampling

Training images that contain a rare class are copied 2 to 4 times (repeat-factor sampling, LVIS, Gupta et al. 2019). For each class c , $r_c = \max(1, \sqrt{t / f_c})$, where f_c is the fraction of train images that contain c and $t = 0.3$; each image is repeated $\text{round}(\max r_c \text{ of its classes})$ times, at most 4. Every box in a copied image stays labelled, so

no vehicle becomes background, and each copy gets its own random augmentation. Validation images are never copied.

Class	Boxes before	Boxes after	×
Car	15,282	21,075	1.4
Motorcycle	5,172	6,922	1.3
Bus	415	564	1.4
Truck	1,072	1,465	1.4
Tuktuk	221	442	2.0
Van	323	648	2.0
Pickup	383	774	2.0
Songthaew	93	372	4.0

Table 2.2 Effect of oversampling on the Round 1 training split (2,388 → 3,178 images).

2.5 Augmentation and input size

Both detectors train and predict at **704 px** (images enlarged 2×) because the objects are tiny. No rotation or vertical flip is used: the cameras are fixed and vehicles are never upside down.

Model	Augmentation
RF-DETR	horizontal flip (p 0.5); brightness/contrast $\pm 20\%$ (p 0.5); hue ± 5 , saturation ± 20 , value ± 15 (p 0.3) for day, night and rain; multi-scale training around 704 px
YOLO26	mosaic (off for the last 10 epochs); horizontal flip (p 0.5); HSV jitter (h 0.015, s 0.7, v 0.4); shift $\pm 10\%$; zoom $\pm 30\%$ (milder than the default, objects are already tiny); random erasing 0.4
Classifier	box randomly shifted up to 10% and resized 85–120% (so it also learns from slightly-off detector boxes); horizontal flip; colour jitter 0.3; label smoothing 0.1

Table 2.3 Augmentation per model.

2.6 Crops for the classifier

The classifier is trained on ground-truth box crops with 15% context on each side, resized to 224 × 224. Rare classes are sampled more often (weight = count^{-0.5}), 12,000 crops per epoch.

2.7 External data

The TA confirmed that external data is allowed. Nine labelled Roboflow datasets of Thai vehicles were collected (about 16,000 unique images after removing Roboflow's pre-made augmented copies). No external image matches any train or test image (perceptual hash).

Dataset	Unique images	Look	Used for
vehicle_car	1,919	Thai CCTV intersections, some close-ups	classifier (+ detector test)
traffic_count	1,219	CCTV, highway; many far vehicles unlabelled	classifier (+ detector test)
vehicle_detection_v22	2,444	close-up photos	classifier
vehicle_detection_v1	1,830	close-up photos	classifier
classification_of_cars	7,173	stock photos, close-up	classifier
songthaew / tuktuk_detection / test_mlejl	505	close-up photos	classifier
bangkok_bus	1,099	phone photos, no labels	not usable

Table 2.4 External datasets.

Each dataset's class names were mapped by hand to the 8 competition classes, following the class definitions and the conventions of our own labels:

- **Kept, examples:** sedan/SUV/PPV/hatchback/taxi → Car; MPV → Van; "truck" in classification_of_cars (actually pickups) → Pickup; "Six-wheeled vehicle bus" (actually 6-wheel trucks) → Truck.
- **Box-pickups → Truck**, because our training labels put pickups with a closed box or canopy under Truck (the class definition includes closed box trucks, รถตู้).
- **Dropped:** bicycles, pedal rickshaws (samlor), motorcycle sidecars (saleng), farm vehicles (e-tan), ambulances, kei trucks, and mixed "3 wheel motorbike" labels.
- **Re-labelled:** traffic_count's mixed labels (Car0 mixes cars and pickups) keep their box and take the class our classifier predicts, limited to the classes they can really be. These boxes are never used to train the classifier.

Result: **12,308 images and 18,867 boxes** in our 8 classes; 16,828 crops are used for the classifier. How they were used, and what helped, is in Chapter 4.4.

3. Model

3.1 Pipeline

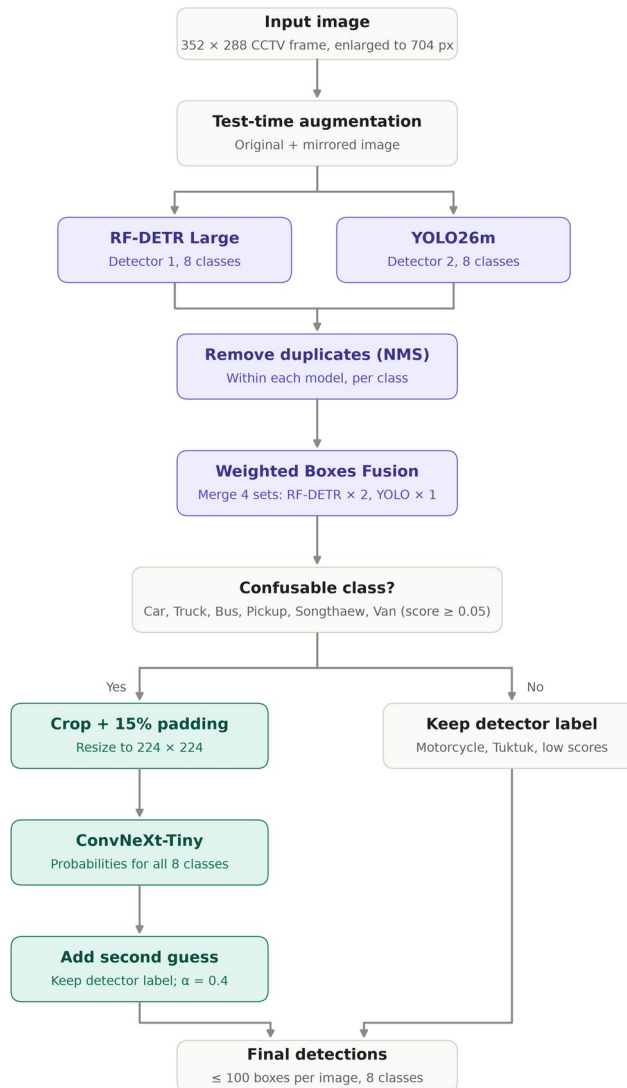


Figure 3.1 Inference pipeline of the final submission.

1. **Two detectors** predict boxes and classes: RF-DETR Large (transformer) and YOLO26m (CNN). They make different mistakes, so combining them helps.
2. **Flip test-time augmentation:** each detector also predicts on the mirrored image, and those boxes are mirrored back.
3. **Duplicate removal:** each model's own overlapping boxes of the same class are removed (NMS, IoU 0.6). Without this, fusion averages weak copies into the good box (measured: 0.456 → 0.351 on a smoke test).
4. **Weighted Boxes Fusion** merges the 4 prediction sets (2 models × normal/mirrored), RF-DETR weighted 2 and YOLO 1.
5. **Crop classifier (second opinion):** ConvNeXt-Tiny re-checks boxes labelled Car, Truck, Bus, Pickup, Songthaew or Van and adds a second, lower-scored class guess (3.4).

Predictions keep scores down to 0.001 and at most 100 boxes per image: mAP ranks boxes by score, so low-scored extra boxes can only add recall.

3.2 Models

	RF-DETR Large	YOLO26m	ConvNeXt-Tiny
Type	transformer detector (DETR family)	single-stage CNN detector	image classifier (CNN)
Parameters	33.6 M	21.8 M (75 GFLOPs)	27.8 M
Pretrained on	COCO (Roboflow release)	COCO (Ultralytics release)	ImageNet (timm)
Input	704 × 704	704 px	box crop, 224 × 224
Library	rfdetr 1.11	ultralytics 8.4	timm

Table 3.1 The three models.

RF-DETR Large. A DETR-style detector: a DINOv2 vision-transformer backbone (windowed "small" variant, patch size 16) extracts features from four depths; a projector turns them into a feature map; a 4-layer decoder with deformable cross-attention (hidden size 256, 8 self-attention and 16 cross-attention heads) refines 300 object queries into boxes and class scores. Training matches queries to objects one-to-one, so it needs no anchors. Its DINOv2 backbone is pretrained on a very large image collection, which helps on unseen cameras.

YOLO26m. The medium model of Ultralytics' YOLO26 family: a convolutional backbone, a feature-pyramid neck and detection heads at three scales. It is fast, strong on small objects, and errs differently from RF-DETR, which is what the ensemble needs.

ConvNeXt-Tiny. A modern CNN with ResNet-like stages redesigned after vision transformers (large 7 × 7 depth-wise convolutions, LayerNorm, GELU). It classifies one enlarged vehicle crop into the 8 classes. Seeing the vehicle large and alone helps with look-alikes such as Pickup and Car.

3.3 Training settings

Setting	RF-DETR Large	YOLO26m	Classifier
Epochs	12	25 (mosaic off last 10)	15 × 12,000 crops
Batch / LR	16 / 1e-4 (backbone 1.5e-4)	16 (nominal 64)	64 / 1e-4, AdamW, wd 0.05
LR schedule	cosine, 1 warm-up epoch	cosine	one-cycle
Weights used	EMA, last epoch	last epoch	last epoch (final)

Table 3.2 Training settings of the final models.

The epoch counts are short on purpose. The final models are trained on all 15 cameras, so there is no clean validation set to pick the best epoch: the last epoch is used. Round 1 showed that 40 RF-DETR epochs peak at epoch 6 and then lose 0.06 mAP50, while 12 epochs end at their best (4.2).

3.4 The classifier adds a second guess

The classifier never overwrites the detector's label. For each checked box, every class c gets the score

$$\text{score}(c) = \text{detector score} \times ((1 - \alpha) \cdot [c = \text{detector label}] + \alpha \cdot p_{\text{classifier}}(c))$$

and a class is written as an extra box when its weight is at least 0.05. With $\alpha = 0.4$, a box the detector calls "Car 0.80" that the classifier sees as 70% Pickup becomes Car 0.58 and Pickup 0.22. mAP ranks boxes by score: a right second guess rescues a rare class, and a wrong one sits low in the ranking and costs almost nothing. $\alpha = 0.4$ was kept after a Kaggle comparison (4.4).

3.5 Generative AI and cost

No generative-AI model or paid API is part of the pipeline: all three models are open-source and run locally. Training and inference used one NVIDIA RTX 5090 (32 GB). One full training round (RF-DETR 10 min, YOLO26m 8 min, classifier 3 min, inference about 1 min) takes about 25 minutes, so the compute cost is GPU time only.

4. Results

Two rounds were run. **Round 1** trains on 12 cameras and measures everything on the 3 held-out validation cameras. **Round 2** retrains the chosen setup on all 15 cameras for Kaggle. Local scores use the same pycocotools mAP@50 as Kaggle.

4.1 What each stage adds (Round 1, validation)

Stage	mAP50	Change
RF-DETR alone	0.634	
+ remove own duplicates	0.639	+0.005
+ flip TTA	0.655	+0.016
+ YOLO26l, WBF 1:1	0.676	+0.021
+ YOLO26l, WBF 2:1	0.683	+0.028
+ 4-class classifier, α 0.4	0.686	+0.003
+ 8-class classifier, α 0.4	0.693	+0.010

Table 4.1 Ablation: each stage was kept only if it raised the validation score.

Ignoring the class, the fused boxes reach 0.92 AP50, against 0.69 with classes: the detectors find almost every vehicle, and most of the loss is the wrong class. That is why the classifier covers all 8 classes.

4.2 Epochs and model size

Run	Best val mAP50 (epoch)	Last epoch
RF-DETR Large, 40 epochs	0.635 (6)	0.577
RF-DETR Large, 12 epochs	0.629 (11)	0.629
RF-DETR Medium, 12 epochs	0.628 (6)	0.608
YOLO26l, 25 epochs	0.624 (13)	0.591
YOLO26m, 25 epochs	0.628 (20)	0.613
YOLO26s, 25 epochs	0.615 (17)	0.597

Table 4.2 Shorter schedules end near their peak, which the final (last-epoch) models need.

4.3 Kaggle submissions

#	File	What changed	Public mAP50
1	wbf_exp_cls_a0.4.csv	Round 1 models; image_id copied from the sample (long Thai names)	0.000
2	wbf_exp_c8big_a0.4_conf05.csv	same, smaller file (still long names)	0.000
3	wbf_exp_c8big_a0.4.csv	short image_id (test file names)	0.637
4	wbf_final_c8big_a0.4.csv	Round 2: all 15 cameras	0.703
5	wbf_final_ext_a0.6.csv	+ external classifier, α 0.6	0.702
6	wbf_final_ext_a0.4.csv	+ external classifier, α 0.4	0.707

Table 4.3 All Kaggle submissions.

Training on all 15 cameras added 0.066: more camera variety is what the unseen test cameras need most. The two 0.000 scores were a format issue, not a model issue (5.2). The final selected submissions are #6 (0.707) and #4 (0.703).

Submissions

Select up to 2 submissions that will count towards your final leaderboard score. If less than 2 are selected, Kaggle will automatically select from your best scoring submissions. [Learn More](#)

1/2

Auto-selection candidates ?

All	Successful	Selected	Errors	Recent ▾
Submission and Description			Public Score ⓘ	Select
✓	wbf_final_ext_a0.4.csv	Complete · 37s ago	0.70700	☐
✓	wbf_final_ext_a0.6.csv	Complete · 19m ago	0.70189	☐
✓	wbf_final_c8big_a0.4.csv	Complete · 1d ago	0.70347	☒

Figure 4.1 Kaggle Submissions page (public scores).

[Replace or add: screenshot of the final leaderboard page.]

4.4 External data

Experiment	Val mAP50	Public	Kept
Baseline (no external)	0.688	0.7035	
A. Classifier + 16,828 external crops, α 0.4	0.705	0.7070	yes
A. Classifier + external crops, α 0.6 (best on val)	0.713	0.7019	no
B. Detectors + 593 external CCTV frames, α 0.4	0.706	-	no

Table 4.4 External-data experiments (Round 1 detectors for validation, Round 2 detectors for Kaggle).

- **A. Classifier.** External crops raised the classifier's balanced accuracy from 67.1% to 69.4% and cut its false alarms on common classes (Car called Pickup: 62 → 20; Motorcycle called Tuktuk: 43 → 3).
- **B. Detectors.** External CCTV images were tiled 4 to a 352×288 frame, so their vehicles match our size (median 4.6% of the frame vs 5.0%), and the vehicles they left unlabelled were filled in by our detectors. The retrained RF-DETR gained alone (0.625 → 0.634) but not after the classifier (0.706 vs 0.705); the retrained YOLO got worse (0.566 → 0.531). Not kept.

4.5 Per-class results (validation)

Class	Without external	With external classifier	Change
Car	0.940	0.941	+0.001
Motorcycle	0.861	0.860	0.000
Bus	0.777	0.786	+0.009
Truck	0.720	0.727	+0.007
Tuktuk	0.849	0.852	+0.003
Van	0.697	0.767	+0.070
Pickup	0.499	0.519	+0.020
Songthaew	0.159	0.191	+0.032
mAP50	0.688	0.705	+0.017

Table 4.5 AP50 per class, full pipeline at α 0.4 on the 3 validation cameras.

5. Discussion

5.1 Error analysis

The weakest classes are Songthaew (AP 0.19) and Pickup (0.52). The main confusions on validation (full pipeline, score ≥ 0.3) were cropped and inspected:

Mistake	Count	What the crops show	Cause
Pickup $\rightarrow$ Car	53 of 114	small pickups seen from above	the cargo bed is hard to see at 352×288
Songthaew $\rightarrow$ Truck	12 of 24	10 are small green pickup-based songthaews at camera 1066	a style missing from training (91 of 117 train Songthaews are from camera 1426)
Truck $\rightarrow$ Songthaew	14	camera 1427: blue 6-wheel trucks with a tarp, often the same vehicle in consecutive frames	genuinely ambiguous from above
Truck $\rightarrow$ Car	24 of 284	white or dark SUVs and pickups from above	label inconsistency: box/canopy pickups are Truck
Car $\rightarrow$ Truck	11	blurry; some box/canopy pickups labelled Car	the same inconsistency in reverse

Table 5.1 Main confusions and their causes.

Labelled Songthaew, predicted Truck (12 shown)



Figure 5.1 Labelled Songthaew, predicted Truck: mostly the small green type at camera 1066.

Songthaew correct (reference) (6 shown)



Figure 5.2 Correctly found Songthaews: the large blue type at camera 1427.

Labelled Truck, predicted Songthaew (14 shown)



Figure 5.3 Labelled Truck, predicted Songthaew: blue tarp trucks at camera 1427.

Labelled Truck, predicted Car (24 shown)



Figure 5.4 Labelled Truck, predicted Car: many look like ordinary SUVs or pickups.



Figure 5.5 A sample of our Truck labels: big trucks, but also box and canopy pickups.

Most remaining Truck errors are label ambiguity, not something more training can fix, and the same ambiguity probably exists in the test labels. Songthaew would need top-view examples of the small green type, which none of the external datasets contain. Many missed motorcycles (236 of 1,391 on validation) are the tiniest objects in the images.

5.2 Lessons from the process

- **Validate like the test.** Holding out whole cameras gave a validation score that ranked choices the same way as Kaggle, and it exposed that long RF-DETR training over-fits the training cameras.
- **Check the scorer's id convention.** The first two submissions scored exactly 0.000 although the file matched the documented format byte for byte. Kaggle's answer key uses the short test file names (as in train.csv), not the long Thai names shown in sample_submission.csv; switching to short names gave 0.637 with the same predictions.
- **A setting tuned on a small validation set can over-fit it.** $\alpha = 0.6$ was best on validation (31 Van and 24 Songthaew boxes) but lower on Kaggle. A one-change comparison (external classifier at the old α 0.4) separated the two effects: external data helped, the higher α hurt.
- **External data helps where the domain matters less.** A cropped vehicle looks alike from any camera, so the classifier gained. The detector learns whole scenes, and the external scenes differ from BMA CCTV, which the test set shares with our training data.

- **Do not chase the public score.** Only about 50% of the test images are public; every decision here was made on validation, and the public score was used as a check.

5.3 How to improve

- **Label more top-view rare vehicles**, especially small green songthaews and pickups, from BMA-style cameras.
- **A consistent rule for box and canopy pickups** (Truck or Pickup) in the training labels.
- **Sliced inference for tiny objects** (predict on overlapping crops at higher zoom) to recover far motorcycles.
- **Pseudo-labelling the test images** with the ensemble and fine-tuning on them (self-training on the new cameras).
- **Pretraining on external data, then fine-tuning on ours**, instead of mixing them, mainly for the classifier.

6. Conclusion

A two-detector ensemble (RF-DETR Large and YOLO26m) with flip TTA, Weighted Boxes Fusion and a ConvNeXt-Tiny second-opinion classifier reaches **0.707 mAP@50** on the public leaderboard, well above the 0.56 full-marks line. The largest gains came from combining two different detectors (+0.03 on validation), training the final models on all 15 cameras (+0.066 public), and an 8-class classifier that adds lower-scored second guesses instead of replacing labels. External data improved the classifier but not the detectors. The remaining errors are mostly look-alike vehicles seen from above (Pickup vs Car, Songthaew vs Truck) and partly inconsistent labels.

Appendix: reproduction and submission package

All commands run from the repository root with the vehicle-det conda environment (environment.yml).
README.md lists every step; the commands that produce the final file are:

```
python src/prepare_data.py --full
python src/train_rfdetr.py --data datasets/rfdetr_full --epochs 12 --name rfdl_e12_full
python src/train_yolo.py --model yolo26m.pt --data datasets/yolo_full/data.yaml --epochs 25 --patience 100 --name y26m_e25_full
python src/predict_detector.py --weights runs/rfdl_e12_full/last_ema.pth --name rfdl_e12_full --splits test
python src/predict_detector.py --weights runs/y26m_e25_full/weights/last.pt --name y26m_e25_full --splits test
python src/fuse.py --runs rfdl_e12_full y26m_e25_full --weights 2 1 --out wbf_final --splits test
python src/prepare_external.py --classifier runs/cls_all8/best.pt
python src/train_classifier.py --full --classes Car Motorcycle Bus Truck Tuktuk Van Pickup Songthaew \
--samples-per-epoch 12000 --external datasets/external/external.csv --name cls_all8_ext_full
python src/reclassify.py --preds wbf_final --classifier runs/cls_all8_ext_full/best.pt \
--apply-to Car Truck Bus Pickup Songthaew Van --alpha 0.4 --splits test --out wbf_final_ext
python src/make_submission.py --preds wbf_final_ext_a0.4
```

Package item	Where
(1) Source code	GitHub: https://github.com/Pongsatorn-K633/vehicle-detection-dsde
(2) Uploaded CSV	wbf_final_ext_a0.4.csv (0.707) and wbf_final_c8big_a0.4.csv (0.703), in this folder and in submissions/
(3) Prepared data	[Google Drive link: datasets/ (yolo_full, rfdetr_full, external)]
(4) Model weights	[Google Drive link: rfdl_e12_full/last_ema.pth, y26m_e25_full/weights/last.pt, cls_all8_ext_full/best.pt]

Table A.1 Submission package. Bracketed items are filled in after upload.